

Attention Is Not Always All You Need

Jack Foreback
College of Computing
Grand Valley State University
forebacj@mail.gvsu.edu

Abstract—Single-cell modeling with AI architectures offers unprecedented advancement in our understanding of the human genome, with applications in the fields of biology, pharmacology, and genetic engineering. While most current work in the field relies heavily on transformer(GPT)-based architectures, our current approach to modeling single-cell transcriptomics data utilizes Variational Autoencoders(VAEs). In this work, we investigate the performance impacts on counts-based cell modeling of adding the core mechanism of the transformer architecture, Multi-Head Self Attention, into the Variational Autoencoder. Through testing linear projections, embeddings, the number of heads, linear layers, and chained modules, we find the optimal structure of the attention module as it applies to our model. We test this structure in all configurations surrounding the latent space of our VAE and discover that implementing Multi-Head Self Attention into our single cell human Variational Autoencoder did not produce significant improvements in the modeling of counts based single-cell data.

I. INTRODUCTION

Transformer-based architectures have gained considerable renown for their success in the fields of Natural Language Processing (NLP) and Computer Vision. Besides NLP models like Chat GPT, other transformer-based models have shown some promise in the field of single-cell modeling, like scGPT [1], and Geneformer [2]. Models like AlphaFold [3], the leader in prediction of protein structures, have also shown transformers’ ability to model complex biological data. In the field of single-cell transcriptomics however, the success of transformer-based models has not been as revolutionary as what was seen in fields like NLP.

Our current approach to single cell modeling utilizes Variational Autoencoders (VAEs), with encoder and decoder layers as well as a latent space modeled by a normal distribution. We measure how well our model performs using Mean Squared Error (MSE), KL-Divergence, UMAPs to visualize our latent space, and SCIB (Single Cell Integration Benchmark) to evaluate integration. We plan to test the attention module’s structure and position, including the number of attention heads, number of linear layers, the addition of a linear projection and embeddings, chaining modules, and locations within the model. This should allow us to find the optimal addition of attention, and observe whether adding a module of Multi-Head Self Attention to our model will significantly improve our performance metrics. We expect to see improvements in our loss metrics and integration as attention should be able to encode additional information about the relationships between data in the compressed latent space of our model.

II. BACKGROUND

A. Variational Autoencoders

Variational Autoencoders(VAEs) are a type of generative AI model which utilize the multi-layer perceptron and back-propagation to encode information and generate data. VAEs are comprised of an encoder and decoder, which use multiple layers with some activation function like ReLU or Sigmoid. The encoder compresses the input data to a latent space, typically modeled by a normal distribution, while the decoder reconstructs the data from this latent representation. VAEs have many nice properties, such as the ability to handle large datasets [4]. The reconstruction loss of our VAE measures how well it was able to reconstruct the cell we feed into it. The error is used to tune the model parameters through backpropagation to improve how well the model can reconstruct our cells. Backpropagation is an algorithm to optimize the parameters in our model to visualize the loss. A more detailed description of backpropagation can be found in [5].

B. 2.2 Attention Mechanism

The attention mechanism, first presented in [6], allows models to selectively focus on different parts of the input data, capturing dependencies in a sequence more effectively than traditional neural networks. It uses key, query, and value matrices to calculate relationships between elements in the input sequence. Each attention head processes the input data independently, capturing different aspects of the sequence, which allows a more comprehensive understanding of the data from multiple perspectives. The outputs of the heads are then concatenated and linearly transformed to match the input dimension of the next layers.

A more detailed description of how attention functions is outlined in [7].

III. METHODS

A. Data

We use a subset of 3 million human cells curated from the Chan Zuckerberg Initiative’s CELLxGENE dataset [8]. The dataset underwent standard preprocessing steps, including quality control and normalization to ensure an equal distribution of cell types. The actual data we feed into the model are vectors of length 60,664, with counts of genes which represent a human cell. Since we have a subset of 3 million of these human cells, we separate them into 15 different .npz files, with .pkl files corresponding to the metadata. Thirteen of these

we feed into the model during the training step, While the fourteenth and fifteen are used for the validation and test step respectively. Each file contains approximately 200,000 cells. Our baseline models and code are all made publicly accessible [9].

B. VAE Architecture

Our encoder contains fully connected layers with ReLU activation and batch-norm, there is a dropout of 0.2 after the 1024 layer. More specifically, our baseline model has an input layer of 60,664, with encoder layers of dimension: 1024, 512, 256. The latent dimension was also 256. The decoder follows a mirrored structure without batch-norm, and with layer dimensions: 256, 512, 1024, 60,664.

Figure 1 illustrates the baseline VAE model, as well as the experimental architecture.

C. Training and Evaluation

The models were trained using Pytorch and the default Adam optimizer with weight decay equal to 10^{-8} [12]. We employed standard evaluation metrics, including reconstruction loss and KL Divergence(KL-Div). Reconstruction loss measures our ability to reconstruct the cell accurately, while KL-Divergence measures how well our model has fit the data to a normal distribution. We use sum reduction to calculate our reconstruction loss and mean reduction to calculate our KL Divergence. We also use UMAPS to visualize our model’s performance. We then compare loss metrics between our baseline VAE and Attention VAE.

IV. RESULTS

V. TESTING ATTENTION METHODS

All initial attention testing was done in the 256 encoder layer, before the latent space. The theory behind this is that if attention can learn relationships at this location, where the data is most compressed, then it should be able to generalize and be added to other locations in the model, where it will also be able to learn. It also lowers training time and computational complexity to have attention calculations performed on smaller matrices.

For our attention block, we use the baseline structure provided in the paper *Attention Is All you Need* [6] for the transformer encoder. Our customized version is available in Figure 2. We also use Pytorch’s built in module for multi-head attention. Due to the nature of our data, we did not use positional encodings. First, we test different implementations of the module structure: Linear Attention, Nonlinear Attention, and Nonlinear Attention with Embeddings. This testing is done with two heads as a baseline. Linear attention has the same structure, but with a linear projection done before feeding our data into Pytorch’s multi-head attention. We found that linear attention performed better than both nonlinear and nonlinear with embeddings, as shown in Table I.

Method	val/loss
Baseline	0.0335
Linear Attention	0.0333
Nonlinear Attention(NLA)	0.0335
NLA with Embeddings	0.0392

TABLE I
ATTENTION METHODS

A. Testing Heads

We then moved on to testing different numbers of heads with linear attention, in powers of 2 up to 128. Two attention heads performed the best, showing the only marked improvement to the model. Validation was run on other attention implementations with the most promising higher attention head candidates, but it was found linear attention still performed the best. Results are shown in Table II.

# of Heads	Validation Loss
0	0.0335
2	0.0333
4	0.0335
8	0.0336
16	0.0335
32	0.0335
64	0.0336
128	0.0335

TABLE II
NUMBER OF ATTENTION HEADS

B. Testing Linear Depth and Chained Modules

Next we tested adding additional linear depth to the linear module following multi-head self attention(refer to Figure 2), and chaining multiple attention modules together with residual connections. Additional linear depth and chaining modules together did not improve the performance of the model, and worsened loss metrics as seen in Table III and Table IV.

# Linear Layers	val/loss	# of Modules	val/loss
2	0.0334	2	0.0334
3	0.0334	3	0.0336
4	0.0334	4	0.0335

TABLE III
ADDING LINEARITY

TABLE IV
CHAINING MODULES

This testing validated our optimal attention module structure as a simple two head implementation of the attention module found in the *Attention Is All You Need* [6].

C. Testing Optimal Locations and Final Validation

We then placed our best performing attention module in all locations surrounding our latent space. We found it performed best in the 512 encoder layer, but even then it did not offer significant enough improvement to consider it worthwhile in adding to our model, as shown in Table V, Table VI, and Table VII. Our SCIB metrics were similarly not significantly improved. It should also be noted that adding too much attention greatly worsened our metrics, which also scrambled our UMAPS into unclustered circles. Initially it was thought this was due to the beta of our model being set to 1. This

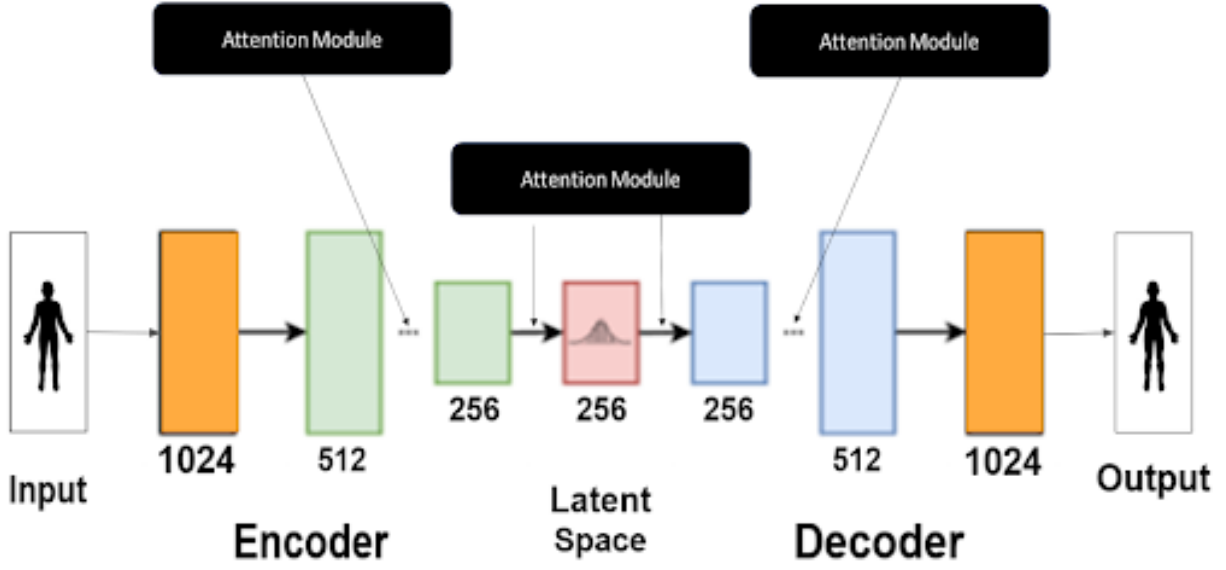


Fig. 1. Experimental Architecture

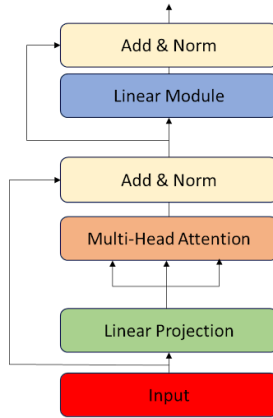


Fig. 2. Module Structure

# Linear Layers	val/loss	# of Modules	val/loss
512 Encoder(En)	0.0332	512 Decoder(De)	0.0535
256 En	0.0332	256 De	0.0535
Both En(512&256)	0.0535	Both De(512&256)	0.0535

TABLE V
ENCODER

TABLE VI
DECODER

Location	val/loss
256 En & 256 De	0.0333
512 En & 512 De	0.0535
Both En & 256 De	0.0535
Both En & 512 De	0.0535
Both De & 256 En	0.0535
Both De & 512 En	0.0535
All Layers	0.0535

TABLE VII
COMBINATIONS

beta orients the loss objective more towards fitting the model to a normal distribution, so we thought we had a problem of overnormalization. We then lowered the beta, but the results continued. This leads us to believe that our VAE is successfully modeling dependencies, so attention has essentially no work left to do in the compressed space, and instead normalizes. To remove this problem we would need to run attention where the data has not been modeled and compressed, so on the length 60,664 input sequence. However, this is not feasible due to computation restrictions. Instead we may need to look at other models like Mamba, which could be able to handle vectors of such large length [12].

VI. CONCLUSION

In this specific application, it does not appear that multi-head self attention offers significant improvement to per-

formance metrics when used in a counts based single-cell transcriptomics autoencoder. The relationships between cell counts may already be successfully modeled in the compressed latent space, leading to attention being unable to extract more meaningful relationships. The compressed nature of the data in that location may have also confounded attention's reasoning ability, but due to compute limitations we could not test larger attention modules. Future research on other methods such as Mamba and conditional layers to improve the model may be needed.

VII. ACKNOWLEDGEMENTS

MMVAE is inspired by works on multi-modal MMVAE from Shi et. al (2019), adversarial integration from Kopp et. al. (2022). Thank you to the student Summer Scholars program and the Office of Undergraduate Research(OURS) at GVSU.

REFERENCES

- [1] Haotian Cui , Chloe Wang, Hassaan Maan , Bo Wang. scGPT: Towards Building a Foundation Model for Single-Cell 2 Multi-omics Using Generative AI. bioRxiv, 2023.
- [2] Zhanbei Cui, Yu Liao, Tongda Xu , Yan Wang. GeneFormer: Learned Gene Compression using Transformer-based Context Modeling. arXiv preprint arXiv:2212.08379, 2022.
- [3] John Jumper, Richard Evans, Alexander Pritzel, Tim Green, Michael Figurnov, Olaf Ronneberger, et al. Highly accurate protein structure prediction with AlphaFold. Nature, 2021.
- [4] D. P. Kingma and M. Welling, "Auto-Encoding Variational Bayes," International Conference on Learning Representations (ICLR), 2014.
- [5] D. E. Rumelhart, G. E. Hinton, and R. J. Williams, "Learning representations by back-propagating errors," Nature, vol. 323, no. 6088, pp. 533-536, 1986.
- [6] Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., and Polosukhin, I., "Attention is all you need," Advances in neural information processing systems, vol. 30, 2017.
- [7] A. Vaswani, "The Annotated Transformer," Harvard University [Online]. Available: <https://nlp.seas.harvard.edu/2018/04/03/attention.html>.
- [8] K. N. Marton et al., "CELLxGENE: A Platform for Interactive Exploration of Large-Scale Single-Cell Gene Expression Data," Chan Zuckerberg Initiative, [Online]. Available: <https://cellxgene.cziscience.com/>.
- [9] Z. DeBruine, "MMVAE," GitHub repository, 2024. [Online]. Available: <https://github.com/zdebruine/MMVAE>.
- [10] PyTorch Team, "PyTorch Documentation," Online. Available: <https://pytorch.org/docs/stable/index.html>.
- [11] Albert Gu and Tri Dao, "Mamba: Linear-Time Sequence Modeling with Selective State Spaces". arXiv:2312.00752, 2024.
- [12] Disclaimer* ChatGPT was used to provide an outline of paper structure and format citations.